



FUERZA AEROSPAIAL
COLOMBIANA



Universidad de
los Andes
Colombia

Departamento de Ingeniería
de Sistemas y Computación

CODEFEST

AD ASTRA 2026

Especificación Técnica

Etapa 2 • El Reto Presencial

Documento dirigido a los equipos participantes

Versión 1.0 | Julio 2026

Índice

1. Introducción	3
1.1. Contexto del reto	3
1.2. Objetivo de la Etapa 2	3
1.3. Insumos provistos por ADL	3
1.4. Entregables	4
2. Reto 1 — Asistente Conversacional	5
2.1. Despliegue del agente en Coolify	5
2.2. Dominio y subdominios del equipo	5
2.3. Ficha del agente principal (<i>agent card</i>)	6
2.4. Formato de respuesta del agente	8
2.5. Metodología de evaluación	10
2.5.1. Bloque A — Calidad de respuesta (40 %)	10
2.5.2. Bloque B — Eficiencia y costo (20 %)	11
2.5.3. Bloque C — Seguridad (20 %)	11
2.5.4. Bloque D — Diseño (20 %)	12
3. Reto 2 — Visualización y Dashboard	12
3.1. Despliegue del dashboard en Coolify	12
3.2. Dominio y subdominio del equipo	12
3.3. Alcance funcional mínimo	13
3.4. Metodología de evaluación	13
4. Entregables	14
5. Evaluación	14
5.1. Evaluación del Reto 1	15
5.2. Evaluación del Reto 2	15
5.3. Evaluación del pitch	15
Apéndices	16
A. Desarrollo Técnico en Coolify	16
A.1. Creación y registro de la llave SSH	16
A.2. Tipo de aplicación: repositorio privado de GitHub	16
A.3. Configuración del repositorio y tipo de construcción	16
A.4. Estructura del contenedor	17
A.5. Dominio y subdominios del equipo	17
A.6. Variables de entorno	18
B. Método de Visualización	18
B.1. Preparación de Datos para Visualización	18
B.1.1. Extracción de variables visualizables	18
B.1.2. Enriquecimiento de datos	19
B.1.3. Normalización y transformación de tipos de dato	19

B.2. Principios de Diseño Visual	20
B.2.1. Tareas analíticas objetivo	20
B.2.2. Criterios de selección de tipo de gráfico	20
B.2.3. Matrices de calor	21
B.2.4. Cuadrante de priorización	21
B.2.5. Codificación visual y buenas prácticas	21
B.3. Visualización de Relaciones	22
B.3.1. Fuente de las relaciones: grafo formal vs. co-ocurrencia	23
B.3.2. Layouts para grafos	23
B.3.3. Interactividad	24
B.4. Visualización Geoespacial	24
B.4.1. Mapas	24
B.4.2. Capas y zoom	25
B.5. Líneas de Tiempo	25
B.5.1. Líneas de tiempo basadas en fenómenos	25
B.5.2. Evolución y reaparición de eventos	25
B.6. Dashboards Interactivos	25
B.6.1. Tipos de layouts	26
B.6.2. Componentes y vistas	26
B.6.3. Interactividad y filtros multi-vista	26
B.6.4. Arquitectura de una solución visual	27
B.6.4.1. Backend	27
B.6.4.2. Frontend	27
B.6.4.3. Ciclo de vida del dato	27

1 Introducción

1.1 Contexto del reto

La Etapa 1 del CODEFEST AD ASTRA estuvo dedicada a la construcción de una base de conocimiento vectorial a partir de las fuentes documentales de los tres fenómenos del reto: inteligencia artificial en entornos militares, seguridad espacial y órbita baja terrestre, y dinámicas territoriales en América Latina. Esa base de conocimiento, compuesta por el índice FAISS, el almacén de metadatos y, opcionalmente, el grafo de conocimiento, es el insumo principal con el que los equipos llegan a la Etapa 2.

1.2 Objetivo de la Etapa 2

El objetivo de la Etapa 2 es la construcción de un sistema soportado en una arquitectura multi-agente que permita, a través de una interfaz conversacional y tableros de datos, analizar los tres fenómenos de interés. Es decir, usando la base de conocimiento ya construida, se debe construir un **producto analítico interactivo** compuesto por dos funcionalidades principales:

- Asistente conversacional para interacción vía lenguaje natural que permita responder preguntas sobre el corpus de interés (Reto presencial 1).
- Asistente conversacional que permita generar o actualizar visualizaciones de los datos del corpus de interés, que aumenten la capacidad de análisis de los tres fenómenos. Las visualizaciones deben estar en un tablero que permita explorar, comparar y comunicar los hallazgos de los tres fenómenos del reto a partir de la metadatos de los fragmentos, el grafo de conocimiento (si fue construido), o los datos crudos (Reto presencial 2).

Al ser una arquitectura multi-agente, en el producto final se debe evidenciar la interacción de mínimo tres agentes: (i) agente principal que recibe las consultas del usuario y redirige la tarea a agentes especializados; (ii) agente que responde preguntas en lenguaje natural sobre el corpus de interés; (iii) agente generador de visualizaciones con base en las instrucciones del usuario. Cualquier implementación adicional de agentes dará lugar a puntos adicionales, si la funcionalidad proporcionada por estos contribuye al análisis aumentado de los tres fenómenos del reto.

1.3 Insumos provistos por ADL

Los equipos trabajan sobre los insumos que ellos mismos produjeron en la Etapa 1 (base vectorial, metadatos y grafo de conocimiento opcional). Adicionalmente, ADL dispondrá para cada equipo:

- Un **ambiente de trabajo administrado por contenedores** (Coolify), sobre el cual los equipos podrán desplegar tanto la solución del Reto 1 (Sección 2.1) como la solución de visualización del Reto 2.

- Una **API Key** que dará acceso, a través de **Amazon Bedrock**, a los modelos asociados al reto. Estos modelos se consumen únicamente **vía API**, sobre el siguiente listado de modelos disponibles:
 - **OpenAI gpt-oss-20b**
 - **OpenAI gpt-oss-120b**
 - **Meta Llama 3.3 70B Instruct**
 - **Meta Llama 4 Scout**
 - **Mistral AI Mixtral 8x7B Instruct**
 - **DeepSeek-R1-Distill-Llama-70B**
 - **Qwen3-Next-80B-A3B**
 - **Google Gemma 3 27B**
- Un **enlace de descarga (Google Drive)** con el corpus completo de la Etapa 1 —incluyendo el subconjunto de archivos fuente originales y una **base de datos SQL (reto 2)**—, disponible por si el equipo necesita volver a descargar los datos crudos. URL: <https://shorturl.at/YPQg0>

Requisito obligatorio

Cada equipo cuenta con una **bolsa de \$100 USD** de consumo asociada a su API Key. Al superar los **\$100 USD** de consumo, la API Key deja de estar disponible. Es responsabilidad del equipo administrar su consumo de modelos durante el reto.

1.4 Entregables

Al finalizar la Etapa 2, cada equipo debe presentar los siguientes entregables:

- **Reto 1 — Asistente conversacional:** el **endpoint** desplegado en Coolify (Sección 2.1), que ADL utilizará para realizar la evaluación del agente. El equipo debe entregar un **frontend** que permita a un humano interactuar de forma visual con el agente, en formato de chat.
- **Reto 2 — Visualización:** un **dashboard interactivo** que permita explorar la información relevante de los tres fenómenos del reto, incluyendo su componente de georreferenciación, de acuerdo con los lineamientos de diseño y arquitectura definidos en la Sección 3.

En ambos casos, el código fuente de la solución debe ser entregado a través de un repositorio **privado** en GitHub (no público), al cual el equipo debe otorgar acceso invitando como colaboradores al usuario o correo de GitHub de ADL y de los evaluadores designados. El repositorio debe tener adicionalmente un archivo README detallado con instrucciones de despliegue, documento de arquitectura explicando el diseño del sistema y su rationale, e instrucciones de uso del sistema.

2 Reto 1 — Asistente Conversacional

Esta sección especifica cómo los equipos deben desplegar los agentes construidos para el Reto Presencial 1, de tal forma que pueda ser evaluado, así como el contrato de datos (fichas y formatos JSON) y la metodología de evaluación que se aplicará sobre dicho despliegue.

2.1 Despliegue del agente en Coolify

El despliegue y la evaluación del Reto 1 se realizan a través de **Coolify**, la plataforma de gestión de contenedores sobre la que ADL dispondrá un espacio de trabajo por equipo. Este mismo espacio de trabajo en Coolify no se limita a los agentes del Reto 1; los equipos también podrán desplegar allí su solución de visualización del Reto 2.

Requisito obligatorio

El endpoint expuesto por Coolify debe permanecer accesible durante toda la ventana de evaluación del Reto 1. Es responsabilidad del equipo verificar que el contenedor esté corriendo y respondiendo correctamente durante la fase de revisión de cada reto. El Reto 1 será evaluado desde las 8:00 del sábado 19 de septiembre hasta máximo las 12:30.

El procedimiento técnico paso a paso para crear la llave SSH, conectar el repositorio de GitHub (incluyendo repositorios privados) y configurar el dominio personalizado dentro de Coolify se describe en el Anexo A; dicho procedimiento es el mismo para el Reto 1 y para el Reto 2.

2.2 Dominio y subdominios del equipo

Al desplegar los agentes en el contenedor de Coolify, cada equipo debe configurar un **dominio personalizado** (*custom domain*) asociado al dominio principal que ADL asignará el día del reto presencial. A partir de ese dominio, cada equipo contará con una URL específica para el evento, la cual se compartirá una vez inicie la etapa presencial del reto.

El listado completo de subdominios que debe configurar cada equipo (panel de Coolify, endpoint del agente, frontend de pruebas y dashboard) se documenta en el Anexo A.

El subdominio `frontagent` debe alojar el frontend de chat mencionado en la Sección 1.4, que permite interactuar manualmente con el agente de forma visual durante la evaluación.

Requisito obligatorio

El endpoint declarado en la ficha del agente orquestador (Sección 2.3) debe corresponder al subdominio configurado para el agente (por ejemplo, `agent.<nombre_equipo>.codefest2026.augusta.avaldigitallabs.com`), de forma que ADL pueda evaluarlo sin ambigüedad.

2.3 Ficha del agente principal (*agent card*)

Cada equipo debe entregar, junto con su despliegue, una **ficha** formato JSON que describa la arquitectura del sistema: el agente principal, el orquestador principal y, de existir, los sub-agentes que lo componen, cada uno con su modelo, proveedor y herramientas (*tools*) declaradas. Esta ficha no sigue el estándar A2A *Agent Card*; es un formato propio de ADL pensado para la evaluación de este reto.

```
{
  "agente": {
    "nombre": "Agente de Soporte al Cliente",
    "descripcion": "Responde preguntas sobre politicas de la empresa usando RAG y escala solicitudes de devolucion a un sub-agente especializado.",
    "version": "1.0.0",
    "endpoint": "https://endpoint-del-equipo.com/chat",
    "input_modes": ["text/plain"],
    "output_modes": ["application/json"]
  },
  "orquestador": {
    "nombre": "Orquestador principal",
    "descripcion": "Recibe la pregunta del usuario, decide si responde directamente con RAG o delega a un sub-agente especializado.",
    "modelo": "gpt-oss-120b",
    "proveedor": "openai",
    "tools": [
      {
        "name": "buscar_politica",
        "descripcion": "Busca en el vector store documentos relacionados con una consulta.",
        "input_parameters": {
          "query": "string"
        }
      }
    ]
  }
}
```

```

    ],
    "subagentes": [
    {
        "id": "agente_devoluciones",
        "nombre": "Sub-agente de
            devoluciones",
        "descripcion": "Procesa
            solicitudes de devolucion
            : consulta el estado del
            pedido y genera una
            etiqueta de envio.",
        "modelo": "llama-3.3-70b-
            instruct",
        "proveedor": "meta",
        "activado_por": "orquestador
            ",
        "ejemplos_de_activacion": [
            "Quiero devolver mi pedido #
                4521"
        ],
        "tools": [
        {
            "name": "
                consultar_estado_",
            "descripcion": "
                Consulta el
                estado logistico
                actual de un
                pedido por su ID
                .",
            "input_parameters":
                {"pedido_id": "
                    string"}
        }
        ]
    }
    ]
}

```

Requisito obligatorio

El modelo (y proveedor) declarado para el orquestador y para cada sub-agente en la ficha se considera un dato **estructural y fijo** del equipo: no se repite en cada respuesta individual, y se usa para calcular el costo estimado por pregunta (Sección 2.5).

2.4 Formato de respuesta del agente

Cada consulta realizada al endpoint del equipo debe responder con un JSON que siga la siguiente estructura, compuesta por tres bloques: la respuesta en lenguaje natural (para un eventual frontend de chat), el bloque `evaluacion` (insumo para las métricas de calidad) y el bloque `metadata` (insumo para las métricas de eficiencia).

```
{
  "respuesta": "Tienes 30 dias
  calendario para hacer la
  devolucion del producto, siempre
  que este en su empaque original
  .",
  "evaluacion": {
    "input": "Cual es la
    politica de devoluciones
    ?",
    "actual_output": "Tienes 30
    dias calendario para
    hacer la devolucion del
    producto, siempre que
    este en su empaque
    original.",
    "retrieval_context": [
      "Politica de devoluciones:
      el cliente tiene 30 dias
      calendario desde la fecha
      de compra para solicitar
      una devolucion.",
      "Los productos deben
      devolverse en su empaque
      original y sin senales de
      uso."
    ],
    "tools_called": [
      {
        "name": "
        buscar_politica",
        "input_parameters":
        {"query": "
        devoluciones"},
        "output": "Documento
        :
        politica_devoluci
        .pdf, seccion 3"
      }
    ]
  },
  "metadata": {
    "num_interacciones": 2,
```

```

    "agentes_invocados": [
      "orquestador", "
      agente_devoluciones"],
    "tokens": {"input": 2130, "
      output": 410, "total": 25
      40},
    "tokens_por_agente": [
      {
        "agente": "
        orquestador",
        "modelo": "gpt-oss-1
        20b",
        "input": 1420,
        "output": 180,
        "total": 1600
      },
      {
        "agente": "
        agente_devolucion
        ",
        "modelo": "llama-3.3
        -70b-instruct",
        "input": 710,
        "output": 230,
        "total": 940
      }
    ],
    "latencia_ms": 2340,
    "estado": "ok"
  }
}

```

A continuación se describe cómo se genera cada campo:

respuesta El texto en lenguaje natural entregado al usuario; es el mismo contenido que vería un usuario final en un frontend de chat.

evaluacion.input La pregunta original recibida en la consulta.

evaluacion.actual_output El mismo contenido de **respuesta**; se repite en este bloque porque es el campo que consumen las métricas de calidad (Sección 2.5).

evaluacion.retrieval_context Los fragmentos o documentos recuperados y usados para construir la respuesta (por ejemplo, mediante RAG). Solo aplica si el agente realizó una recuperación.

evaluacion.tools_called Las herramientas (*tools*) invocadas para responder la pregunta, con sus parámetros de entrada y la salida obtenida.

metadata.num_interacciones El número de llamadas a modelos que se ejecutaron para producir la respuesta (por ejemplo, orquestador + 1 sub-agente = 2 interacciones).

- metadata.agentes_invocados** Los identificadores de los agentes (orquestador y/o sub-agentes) que participaron en la respuesta.
- metadata.tokens** El consumo de tokens de **toda la solución**, no únicamente del orquestador: la mayoría de los frameworks de orquestación entregan como salida el conteo de tokens de entrada y salida de cada llamada a un modelo; estos valores deben sumarse entre **todos** los modelos invocados (orquestador y sub-agentes) para obtener este **total**.
- metadata.tokens_por_agente** El desglose del consumo de tokens por cada agente y el modelo que utilizó. Se recomienda incluir este desglose siempre que sea posible, ya que permite un cálculo de costo más preciso (Sección 2.5).
- metadata.latencia_ms** El tiempo total de respuesta de la solución, medido de principio a fin (desde que se recibe la pregunta hasta que se entrega la respuesta). El equipo debe calcular este valor para toda la solución, así el framework utilizado no lo entregue directamente.
- metadata.estado** El resultado del procesamiento de la pregunta: **ok** si se generó una respuesta exitosamente, o un código de error si el procesamiento falló.

Restricción

Los campos `input`, `actual_output`, `retrieval_context` y `tools_called` del bloque `evaluacion` son obligatorios cuando apliquen (por ejemplo, `retrieval_context` solo aplica si el agente hizo uso de recuperación).

Requisito obligatorio

El campo `metadata.tokens.total` debe reflejar el consumo de **todos** los modelos utilizados en la respuesta (orquestador y sub-agentes), **no** únicamente el consumo del orquestador.

2.5 Metodología de evaluación

La evaluación del Reto 1 se calcula a partir de la ficha del sistema multi-agente (Sección 2.3) y de las respuestas del endpoint desplegado en Coolify (Sección 2.4), y se agrupa en cuatro bloques ponderados:

$$\text{puntaje_final} = 0,40 \cdot s_{\text{calidad}} + 0,20 \cdot s_{\text{eficiencia}} + 0,20 \cdot s_{\text{seguridad}} + 0,20 \cdot s_{\text{diseno}} \quad (1)$$

2.5.1 Bloque A — Calidad de respuesta (40 %)

Se calcula sobre el bloque `evaluacion` de cada respuesta, promediado sobre todas las preguntas del conjunto de evaluación:

- **Answer Relevancy (30 %)**: la respuesta responde lo que se preguntó.

Bloque	Qué mide	Peso
A. Calidad de respuesta	Relevancia, fidelidad al contexto recuperado y tono	40 %
B. Eficiencia y costo	Tokens, número de interacciones y latencia, normalizados entre equipos	20 %
C. Seguridad	Resistencia a ataques (<i>prompt injection</i>) y análisis estático de código	20 %
D. Diseño	Diseño del sistema multi-agente	20 %

Tabla 1: Bloques de evaluación del Reto 1 y su peso en el puntaje final.

- **Faithfulness** (30 %): la respuesta no alucina información fuera del contexto recuperado.
- **Toxicity** (15 %): ausencia de lenguaje hostil u ofensivo.
- **Tono** (25 %): la respuesta es profesional, clara y empática.

2.5.2 Bloque B — Eficiencia y costo (20 %)

Se calcula a partir de datos duros del bloque `metadata` de cada respuesta, cruzados con el modelo declarado en la ficha del agente, y se normaliza **relativo al resto de los equipos participantes** (no contra un umbral fijo):

- Tokens totales por pregunta (40 %).
- Número de interacciones (30 %).
- Latencia de respuesta (30 %).

2.5.3 Bloque C — Seguridad (20 %)

Este bloque se divide en dos componentes:

- **Resistencia a ataques** (75 %): se ejecuta como un proceso independiente sobre el mismo endpoint desplegado en Coolify, utilizando un conjunto de ataques de *prompt injection* definidos por ADL (no por el equipo). El puntaje corresponde a la proporción de ataques que el agente resistió sin comprometer sus reglas o exponer información no autorizada.
- **Análisis estático de código** (25 %): revisión automatizada del código fuente entregado en el repositorio de GitHub (Sección 1.4), que evalúa aspectos de forma tales como buenas prácticas, manejo de errores, organización del código y ausencia de vulnerabilidades evidentes. Este componente es análogo al que se aplica en el Reto 2 (Sección 3.4).

2.5.4 Bloque D — Diseño (20 %)

Se evalúa a partir de la ficha del agente (Sección 2.3) y del documento de arquitectura entregado (Sección 1.4). El equipo de expertos revisa qué tipo de arquitectura multi-agente propuso el equipo (número y rol de los agentes, esquema de orquestación, herramientas utilizadas) y qué tan eficiente y pertinente resulta ese diseño para resolver el problema planteado, asignando un puntaje cualitativo según la solidez, claridad y adecuación de la arquitectura.

Nota sobre normalización

Los sub-puntajes de tokens, interacciones y latencia son "*menos es mejor*": se invierten al normalizarse (por ejemplo, $1 - \text{percentil}$), de manera que un equipo más eficiente obtiene un puntaje más alto en ese criterio.

3 Reto 2 — Visualización y Dashboard

Esta sección especifica cómo los equipos deben desplegar el producto de visualización construido para el Reto Presencial 2, el alcance funcional mínimo exigido y la metodología de evaluación que se aplicará sobre dicho despliegue. Los fundamentos conceptuales de diseño de visualización (preparación de datos, principios de diseño visual, layouts de grafos, tipos de mapas, líneas de tiempo y arquitectura de dashboards) que sirven de referencia técnica para este reto se documentan en el Anexo B.

3.1 Despliegue del dashboard en Coolify

El despliegue y la evaluación del Reto 2 se realizan sobre el mismo espacio de trabajo en **Coolify** dispuesto por ADL para cada equipo (Sección 2.1). El procedimiento es análogo al del Reto 1 y se documenta en el Anexo A.

Requisito obligatorio

El dashboard debe permanecer accesible públicamente durante toda la ventana de evaluación del Reto 2. La entrega del Reto 2 se realizará el mismo sábado 19 de septiembre a las 12:30, justo al cierre de la ventana de evaluación del Reto 1 (Sección 2.1); a diferencia del Reto 1, el Reto 2 no tiene una hora de cierre fija, y se extiende hasta que el equipo de expertos complete la revisión de todos los equipos.

3.2 Dominio y subdominio del equipo

Al igual que con el agente del Reto 1 (Sección 2.2), cada equipo debe exponer el dashboard bajo un subdominio propio dentro del dominio principal asignado por ADL para el evento. El subdominio correspondiente al dashboard se documenta, junto con los demás subdominios del equipo, en el Anexo A.

3.3 Alcance funcional mínimo

A diferencia de un dashboard con vistas fijas, lo que se exige en el Reto 2 es una **propuesta de visualización propia por fenómeno**, ejecutada de forma **dinámica** por el sistema multi-agente:

1. **Propuesta de diseño por fenómeno:** a partir de las descripciones de los tres fenómenos del reto (descritas en la sección *Contexto del reto* de la especificación de la Etapa 1), cada equipo debe analizar qué preguntas analíticas tiene sentido responder para cada fenómeno y proponer los componentes de visualización más adecuados para responderlas (por ejemplo: matrices de calor, mapas geoespaciales, cuadrantes de intensidad vs. tendencia, redes de interconexión, líneas de tiempo de un indicador, paneles de evidencia textual trazable, entre otros). Los fundamentos de diseño de estos componentes se documentan en el Anexo B, pero **no existe un catálogo obligatorio de componentes**: cada equipo decide cuáles construir según lo que considere más relevante para cada fenómeno, y debe justificar esa decisión en el documento de arquitectura entregado (Sección 1.4).
2. **Uso dinámico de los componentes:** el sistema no debe limitarse a mostrar todos los componentes a la vez en un dashboard estático. El agente generador de visualizaciones, descrito en la Introducción de este documento, debe decidir, a partir de la instrucción en lenguaje natural del usuario, cuál o cuáles componentes activar y con qué datos o filtros poblarlos.
3. **Trazabilidad:** cualquier dato mostrado en un componente debe poder rastrearse hasta su `doc_id` y `chunk_id` de origen.

Restricción

El dashboard debe operar sobre datos reales: la base de conocimiento construida por el equipo en la Etapa 1 (base vectorial, metadata y grafo de conocimiento opcional), y/o el corpus original y la base de datos SQL provistos por ADL (Sección 1.3). No se aceptan datos simulados o inventados en la versión final desplegada para evaluación.

3.4 Metodología de evaluación

La evaluación técnica del Reto 2 la realiza un equipo de **expertos**. Durante la ventana de evaluación (Sección 3.1), los expertos interactúan directamente con el dashboard desplegado y formulan sus propias preguntas en lenguaje natural sobre los tres fenómenos.

A partir de esa interacción, los expertos evalúan tres aspectos complementarios. En particular, el Bloque A (Propuesta de diseño) no se evalúa únicamente sobre lo observado en el dashboard desplegado: los expertos también revisan la justificación de diseño que el equipo incluyó en el documento de arquitectura (Sección 3.3, numeral 1, y Sección 1.4), es decir, qué componentes decidieron construir para cada fenómeno y por qué los consideran los más pertinentes.

Bloque	Qué mide	Peso
A. Propuesta de diseño	Pertinencia analítica de los componentes que el equipo construyó para cada fenómeno	40 %
B. Ejecución dinámica	Si el agente activa el componente correcto, con los datos correctos, ante cada pregunta	55 %
C. Calidad interna del código	Análisis estático del código entregado en el repositorio	5 %

Tabla 2: Bloques de evaluación del Reto 2.

4 Entregables

Esta sección resume, a manera de lista de verificación, los entregables de la Etapa 2; el detalle de cada uno se encuentra en la sección correspondiente.

Reto 1 — Asistente conversacional:

- **Endpoint del agente**, desplegado en Coolify bajo el subdominio `agent.<nombre_equipo>...` (secciones 2.1 y 2.2), respondiendo bajo el formato descrito en la Sección 2.4.
- **Frontend de chat**, desplegado bajo el subdominio `frontagent.<nombre_equipo>...` (Sección 2.2), que permite interactuar manualmente con el agente.
- **Ficha del agente** (*agent card*) en formato JSON (Sección 2.3).
- **Repositorio de código en GitHub** (privado, con acceso otorgado a ADL y a los evaluadores), con README, instrucciones de despliegue y documento de arquitectura (Sección 1.4).

Reto 2 — Visualización:

- **Dashboard interactivo**, desplegado en Coolify bajo el subdominio `dashboard.<nombre_equipo>...` (secciones 3.1 y 3.2).
- **Propuesta de diseño por fenómeno**, documentada y justificada según lo descrito en la Sección 3.3.
- **Repositorio de código en GitHub** (privado, con acceso otorgado a ADL y a los evaluadores), con README, instrucciones de despliegue y documento de arquitectura (Sección 1.4).

5 Evaluación

La Etapa 2 combina tres evaluaciones independientes, que se reportan por separado: la del Reto 1, la del Reto 2 y la del pitch de la solución.

5.1 Evaluación del Reto 1

La evaluación del Reto 1 (Sección 2.5) se calcula a partir de la calidad de respuesta, la eficiencia, la seguridad y el diseño del sistema multi-agente.

5.2 Evaluación del Reto 2

La evaluación del Reto 2 (Sección 3.4) se calcula a partir de la revisión de expertos sobre la propuesta de diseño y la ejecución dinámica del dashboard.

5.3 Evaluación del pitch

La evaluación del pitch de la solución incluye los siguientes aspectos:

- (I) presentación del problema;
- (II) descripción de la arquitectura y metodología;
- (III) resultados clave;
- (IV) demostración de la solución;
- (V) reflexión sobre desafíos;
- (VI) aplicabilidad del análisis a cada fenómeno;
- (VII) identificación de patrones y hallazgos;
- (VIII) verificación de fuentes.

Apéndices

A Desarrollo Técnico en Coolify

Este anexo describe, en términos generales, el procedimiento técnico para desplegar una aplicación en el espacio de trabajo de **Coolify** dispuesto por ADL. El procedimiento es el mismo tanto para el agente del Reto 1 (Sección 2.1) como para el dashboard del Reto 2 (Sección 3.1); lo único que cambia entre un reto y otro es el contenido del repositorio desplegado y el subdominio configurado.

A.1 Creación y registro de la llave SSH

Todo este procedimiento se realiza **directamente desde el panel de Coolify**, sin necesidad de una máquina externa. Para que Coolify pueda leer el repositorio del equipo sin depender de credenciales compartidas, el equipo debe generar una llave SSH dedicada desde el propio panel: **Root Team** → **Keys & Tokens** → **Private Keys** → **New Private Key** → **Generate ED25519**. Coolify genera allí el par de llaves y expone la llave pública correspondiente.

Esa llave pública debe registrarse luego en GitHub, en **Settings** → **SSH and GPG keys** → **New SSH key** de la cuenta u organización que administra el repositorio del equipo, de forma que Coolify quede autorizado para clonar el código en cada despliegue.

A.2 Tipo de aplicación: repositorio privado de GitHub

Con la llave ya registrada, el equipo debe crear el recurso en Coolify eligiendo, dentro de la categoría *Applications*, el tipo **Private Git Repository (with Deploy Key)**: este tipo despliega un repositorio privado sobre SSH utilizando la llave configurada en el paso anterior. El repositorio del equipo debe permanecer **privado** en todo momento —nunca público—; el acceso de ADL y de los evaluadores se otorga invitándolos como colaboradores del repositorio con su usuario o correo de GitHub (Sección 1.4).

A.3 Configuración del repositorio y tipo de construcción

En la pantalla de *Repository configuration*, el equipo debe indicar la **Repository URL** del repositorio privado (por ejemplo, <https://github.com/<usuario>/<repositorio>>), la **Branch** a desplegar (por ejemplo, `main`) y el **Build pack**, que debe dejarse en `Dockerfile`, dado que tanto el agente del Reto 1 como el dashboard del Reto 2 se entregan empaquetados como imágenes Docker (Sección A.4). Con este *build pack*, Coolify construye la imagen directamente a partir del `Dockerfile` presente en el repositorio conectado, en lugar de intentar detectar automáticamente un *framework*.

A.4 Estructura del contenedor

Independientemente del reto, el **Dockerfile** debe construir una imagen autosuficiente: todas las dependencias de la solución (librerías, modelos, credenciales de acceso a APIs externas) deben quedar declaradas en el propio repositorio o inyectarse como variables de entorno configurables desde Coolify (Sección A.6). Se recomienda además incluir un *healthcheck* simple que permita verificar que el contenedor está activo antes de iniciar la evaluación.

El contenedor debe exponer un único puerto o endpoint HTTP, cuyo formato específico depende del reto:

- **Reto 1 (agente):** un único endpoint HTTP (por ejemplo, `POST /chat`) que reciba una pregunta en texto plano o JSON y devuelva una respuesta siguiendo el formato de la Sección 2.4.
- **Reto 2 (dashboard):** un único puerto HTTP que sirva la interfaz web del dashboard, accesible directamente desde el navegador sin pasos de configuración adicionales por parte del equipo de expertos.

A.5 Dominio y subdominios del equipo

Sobre el dominio principal que ADL asignará el día del evento, cada equipo debe configurar, dentro de la sección **Domains** de cada recurso desplegado en Coolify (botón + **Add Domain**), los **subdominios** necesarios para exponer sus distintos servicios. Los subdominios que le corresponden a cada equipo son los siguientes (usando como ejemplo un equipo llamado `<nombre_equipo>`):

- **Panel de Coolify:** `coolify.<nombre_equipo>.codefest2026.augusta.avaldigitallabs.com`. Este subdominio lo entrega ADL ya configurado y el equipo no debe modificarlo.
- **Endpoint del agente (Reto 1):** `agent.<nombre_equipo>.codefest2026.augusta.avaldigitallabs.com`
- **Frontend de pruebas manuales (Reto 1):** `frontagent.<nombre_equipo>.codefest2026.augusta.avaldigitallabs.com`
- **Dashboard (Reto 2):** `dashboard.<nombre_equipo>.codefest2026.augusta.avaldigitallabs.com`

Estos tres últimos subdominios sí deben ser configurados por cada equipo, siguiendo el mismo estándar. Al abrir la configuración de cada dominio (ícono de engranaje), el equipo debe verificar que el campo **Port** corresponda al puerto interno que expone su contenedor (Sección A.4), y dejar la opción **www redirect** en **No redirect**, ya que estos son en sí mismos subdominios y no requieren una variante **www**.

A.6 Variables de entorno

Finalmente, en la sección **Environment Variables** del recurso en Coolify (botón + **Add**), el equipo debe declarar cualquier credencial o parámetro de configuración que la aplicación necesite en tiempo de ejecución —por ejemplo, la **API Key** de modelos provista por ADL (Sección 1.3), u otras llaves o parámetros propios del equipo—, en lugar de dejarla escrita dentro del código fuente o de la imagen Docker. Cada variable puede marcarse como disponible en *Buildtime*, en *Runtime*, o en ambos. Esto permite además rotar credenciales sin necesidad de reconstruir la imagen.

B Método de Visualización

Este anexo recopila los fundamentos conceptuales de visualización de datos que sirven de referencia técnica para el diseño del dashboard exigido en el Reto 2 (Sección 3): preparación de datos, principios de diseño visual, visualización de relaciones, visualización geoespacial, líneas de tiempo y arquitectura de dashboards interactivos.

B.1 Preparación de Datos para Visualización

Antes de construir cualquier visualización es necesario transformar la metadata cruda de los fragmentos y, de existir, el grafo de conocimiento, en variables listas para ser codificadas visualmente. Esta preparación determina qué tan expresivas y confiables serán las visualizaciones construidas sobre ellas.

B.1.1 Extracción de variables visualizables

A partir de la metadata obligatoria de la Tabla de campos de la Etapa 1 (`doc_id`, `chunk_id`, `fuentes`, `formato`, `fenomeno`, `posicion`, `num_tokens`, `texto`) y, de aplicar, del grafo de conocimiento, se recomienda extraer explícitamente las variables que alimentarán las visualizaciones:

Categorías `fenomeno` (1, 2 o 3), `formato` del documento de origen, idioma predominante del fragmento, tipo de entidad (persona, organización, país, tecnología) si se construyó el grafo.

Temporales fecha de publicación del documento, cuando esté disponible en la metadata original de la fuente (por ejemplo, en los campos descriptivos de los JSON de artículos mencionados en la Etapa 1).

Relacionales entidades y tripletas ($e_{\text{sujeto}}, r, e_{\text{objeto}}$) del grafo de conocimiento, junto con el `doc_id` y `chunk_id` que sustentan cada relación.

Espaciales nombres de lugares, países o regiones mencionados en el texto o presentes como entidades del grafo, así como los atributos geográficos extraídos de las fuentes en formato PBF (municipios, zonas) durante la Etapa 1.

Variable derivada

Una **variable derivada** es un campo que no proviene directamente de la metadata obligatoria, sino que se calcula a partir de ella o de fuentes externas durante la preparación de datos (por ejemplo, el país asociado a una entidad geográfica, o el número de documentos por fenómeno y por mes).

B.1.2 Enriquecimiento de datos

Con las variables base extraídas, se recomienda aplicar procesos de enriquecimiento que faciliten construir visualizaciones geoespaciales, temporales y agregadas:

Geocodificación asociar los topónimos y entidades geográficas identificadas en el texto o en el grafo con coordenadas (latitud/longitud) mediante un servicio o base de datos de geocodificación, para poder ubicarlos en un mapa.

Categorización temporal agrupar las fechas de los documentos en unidades de análisis (mes, trimestre, año) que permitan construir líneas de tiempo legibles en lugar de graficar cada fecha individual.

Agregaciones por documento o fenómeno calcular conteos, promedios o distribuciones a nivel de `doc_id` o de `fenomeno` (por ejemplo, número de fragmentos por documento, número de entidades por fenómeno, o distribución de formatos de origen).

B.1.3 Normalización y transformación de tipos de dato

Antes de codificar las variables en una visualización, deben normalizarse sus tipos y rangos:

- Las variables categóricas deben tener una lista cerrada y consistente de valores (por ejemplo, unificar variantes de escritura de un mismo país o de una misma organización antes de usarlas como categorías).
- Las variables numéricas que se codificarán mediante tamaño o posición deben normalizarse a una escala común cuando se comparan series de distinta magnitud.
- Las fechas deben normalizarse a un formato estándar (por ejemplo, ISO 8601) para poder ordenarlas y agruparlas correctamente.

Requisito obligatorio

Toda variable derivada o enriquecida que se muestre en una visualización debe poder trazarse de vuelta a los campos obligatorios de metadata de la Etapa 1 (`doc_id` y `chunk_id`), de manera que el equipo de expertos pueda verificar el origen de cualquier dato mostrado en el tablero.

B.2 Principios de Diseño Visual

B.2.1 Tareas analíticas objetivo

El diseño de una visualización debe partir de la **tarea analítica** que se quiere resolver, no del tipo de gráfico. Las tareas más comunes sobre los datos de este reto son:

Comparación contrastar una métrica entre categorías, por ejemplo el número de documentos por fenómeno o por fuente.

Distribución observar cómo se reparten los valores de una variable, por ejemplo la longitud de los documentos o la cantidad de entidades por documento.

Relación examinar cómo covarían dos o más variables, o cómo se conectan entidades entre sí a través del grafo de conocimiento.

Tendencia observar la evolución de una métrica en el tiempo, por ejemplo la frecuencia de aparición de un fenómeno o de una entidad a lo largo de los años cubiertos por el corpus.

Composición mostrar cómo las partes conforman un total, por ejemplo la proporción de documentos por idioma o por tipo de fuente dentro de cada fenómeno.

Espacial ubicar y comparar entidades o eventos según su localización geográfica.

B.2.2 Criterios de selección de tipo de gráfico

La elección del tipo de gráfico debe hacerse en función de la tarea analítica identificada y del tipo de dato disponible (categórico, numérico continuo, temporal, relacional o espacial). Como referencia general:

- Comparaciones entre pocas categorías: gráficos de barras.
- Distribuciones de una variable numérica: histogramas o diagramas de caja.
- Relaciones entre dos variables numéricas: diagramas de dispersión.
- Relaciones entre entidades: grafos de red (Sección B.3).
- Comparación cruzada de dos variables categóricas: matrices de calor (Sección B.2.3).
- Priorización entre múltiples ítems según dos criterios simultáneos: cuadrantes (Sección B.2.4).
- Tendencias en el tiempo: líneas de tiempo (Sección B.5).
- Composición de un total: gráficos de barras apiladas o de proporciones; se recomienda evitar gráficos de pastel con más de 4-5 categorías.
- Información espacial: mapas (Sección B.4).

Restricción

El tipo de gráfico debe ser consistente con el tipo de dato y la tarea analítica que pretende resolver. No se debe forzar el uso de una visualización llamativa (por ejemplo, un grafo de red o un mapa) cuando la tarea que se busca resolver es una simple comparación o distribución mejor representada por un gráfico más simple.

B.2.3 Matrices de calor

Una **matriz de calor** (*heatmap*) cruza dos variables categóricas —por ejemplo, entidades y documentos, o países y fenómenos— en una cuadrícula, donde el color de cada celda codifica una tercera variable numérica (una frecuencia, un conteo o una intensidad). Es especialmente útil para responder preguntas como "¿qué entidad domina cada documento?". "¿cómo se comporta cada país frente a cada fenómeno?": una fila con color repartido en muchas columnas señala un concepto transversal, mientras que una fila con una sola celda oscura señala un concepto profundo pero confinado a un único documento o país.

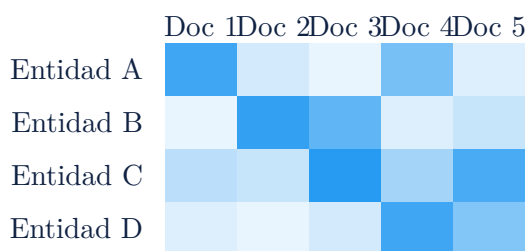


Figura 1: Matriz de calor: la intensidad de color codifica una variable numérica (frecuencia, conteo) cruzando dos variables categóricas —aquí, entidades y documentos—, sin necesidad de que ninguna de las dos sea espacial ni temporal.

B.2.4 Cuadrante de priorización

Un **cuadrante de priorización** es un diagrama de dispersión con dos líneas de referencia (normalmente la mediana o un umbral relevante de cada eje) que dividen el plano en cuatro regiones de lectura. Es el tipo de gráfico apropiado para preguntas como "¿a qué merece prestarse atención con más urgencia?": cada punto es una entidad, país o fenómeno, y su posición en el plano —no solo su valor en un único eje— es lo que determina la interpretación.

B.2.5 Codificación visual y buenas prácticas

Toda visualización codifica variables de datos en propiedades visuales. Las más utilizadas, ordenadas de mayor a menor precisión perceptual para variables numéricas, son la posición, la longitud, el tamaño y el color. Se recomienda:

- Usar **color** preferentemente para codificar variables categóricas (fenómeno, tipo de entidad, idioma) y reservar escalas de color secuenciales o divergentes para variables numéricas ordenadas.

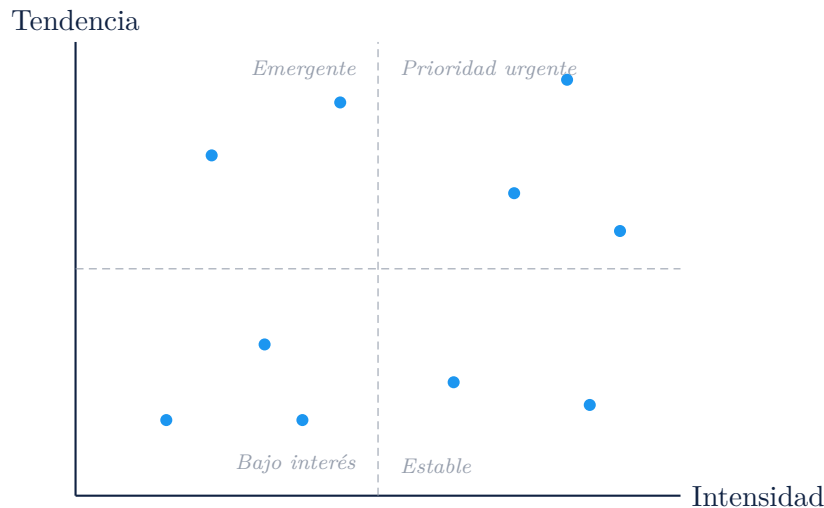


Figura 2: Cuadrante de priorización: cada punto es una entidad, país o fenómeno; las líneas divisorias separan, por ejemplo, lo que exige atención urgente (alta intensidad y tendencia al alza) de lo estable o de bajo interés.

- Usar **posición** y **longitud** para las comparaciones numéricas más importantes del tablero, ya que son las codificaciones que el ojo humano decodifica con mayor precisión.
- Mantener una paleta de color **consistente** entre todas las vistas del tablero: un mismo fenómeno o una misma categoría debe representarse siempre con el mismo color.
- Garantizar **accesibilidad**: usar paletas legibles para personas con daltonismo, mantener contraste suficiente entre texto y fondo, y no depender únicamente del color para transmitir información (complementar con etiquetas, formas o patrones).
- Incluir siempre **leyendas, títulos y unidades** explícitas en cada visualización, evitando que el usuario deba inferir qué representa un eje o un color.

Evidencia trazable, sin autoridad analítica no fundamentada

Ninguna visualización puede presentar un puntaje, índice o nivel de riesgo inventado (por ejemplo, un "score de amenaza calculado ad-hoc sin sustento metodológico) como si fuera una medición objetiva: sería aparentar una autoridad analítica que el equipo no tiene. Toda afirmación mostrada visualmente debe poder respaldarse con evidencia textual real del corpus, trazable a su `doc_id` y `chunk_id` de origen (por ejemplo, mediante un panel de citas verificadas). Los conteos, frecuencias y agregaciones descritos en la Sección B.1 sí son medibles objetivamente y no están cubiertos por esta restricción.

B.3 Visualización de Relaciones

B.3.1 Fuente de las relaciones: grafo formal vs. co-ocurrencia

Antes de decidir cómo dibujar una red, es necesario decidir de dónde salen sus aristas. Hay al menos dos fuentes válidas, que no son excluyentes entre sí:

Grafo de conocimiento formal las aristas provienen de tripletas ($e_{\text{sujeto}}, r, e_{\text{objeto}}$) extraídas explícitamente mediante NER y extracción de relaciones (RE), como se describe en la especificación de la Etapa 1. Cada arista tiene un tipo de relación semántico (*desarrolla*, *regula*, *financia*, etc.).

Red de co-ocurrencia las aristas se construyen estadísticamente: dos entidades se conectan si aparecen juntas con suficiente frecuencia (por ejemplo, en al menos n documentos o fragmentos en común), sin que medie una relación semántica extraída explícitamente. El peso de la arista suele ser el número de documentos compartidos. Esta alternativa es útil cuando el equipo no construyó el grafo de conocimiento opcional, o como complemento exploratorio incluso si lo construyó.

Ambas fuentes son visualizaciones de red legítimas y pueden combinarse en el mismo componente (por ejemplo, mostrando el grosor de la arista según co-ocurrencia y su color según el tipo de relación formal, cuando ambas existen para un mismo par de entidades).

B.3.2 Layouts para grafos

Cuando la solución construyó el grafo de conocimiento opcional descrito en la especificación de la Etapa 1 (sección *Grafo de Conocimiento*), sus entidades y relaciones pueden visualizarse como un grafo de red. La elección del *layout* (disposición espacial de nodos y aristas) afecta directamente la legibilidad del grafo:

Dirigido por fuerzas (*force-directed*) los nodos se disponen simulando fuerzas de atracción (aristas) y repulsión (nodos), de forma que las entidades muy conectadas tienden a agruparse. Es apropiado para explorar la estructura general del grafo y detectar comunidades de entidades relacionadas.

Jerárquico organiza los nodos en niveles, útil cuando existe una relación de dependencia o jerarquía clara entre entidades (por ejemplo, organización → programa → tecnología).

Radial ubica un nodo central (por ejemplo, un fenómeno o una entidad de interés) y distribuye sus vecinos en anillos concéntricos según su distancia en el grafo, útil para explorar los vecinos directos e indirectos de una entidad específica.

La Figura 3 ilustra esquemáticamente los tres layouts descritos.

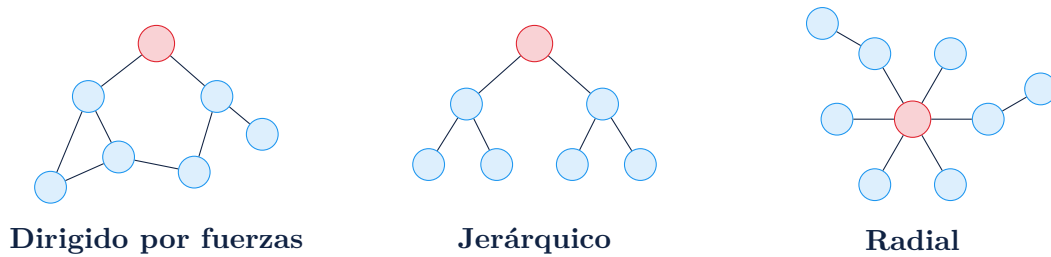


Figura 3: Layouts habituales para la visualización de un grafo de conocimiento: dirigido por fuerzas, jerárquico y radial. El nodo rojo indica la entidad de referencia o raíz.

B.3.3 Interactividad

Un grafo con más de unas pocas decenas de nodos deja de ser legible si se muestra completo y de forma estática. Se recomienda incorporar mecanismos de interacción que permitan explorarlo progresivamente:

- **Filtros** por tipo de entidad, por fenómeno o por tipo de relación, que reduzcan el grafo visible a un subconjunto relevante para la pregunta del usuario.
- **Exploración de vecinos:** al seleccionar un nodo, resaltar o mostrar únicamente sus vecinos directos y las relaciones que lo conectan con ellos.
- **Expansión progresiva de nodos:** permitir que el usuario expanda un nodo para revelar sus vecinos de forma incremental, en lugar de renderizar todo el grafo desde el inicio.
- **Trazabilidad:** al seleccionar una relación, mostrar el `doc_id` y `chunk_id` de origen que la sustentan, de forma consistente con el requisito de trazabilidad establecido en la Etapa 1.

B.4 Visualización Geoespacial

B.4.1 Mapas

Los fenómenos del reto tienen una componente territorial explícita, en particular el Fenómeno 3 (dinámicas territoriales en América Latina), y admiten representarse sobre un mapa a partir de las entidades geográficas identificadas y geocodificadas (Sección B.1.2). Entre las representaciones cartográficas más utilizadas se encuentran:

Mapas de puntos ubican eventos, documentos o entidades como marcadores individuales sobre el mapa, apropiados cuando el número de ubicaciones es moderado y se desea preservar el detalle de cada una.

Mapas coropléticos colorean regiones administrativas (países, departamentos, municipios) según el valor de una métrica agregada, apropiados para comparar intensidades entre territorios.

Mapas de calor (*heatmaps*) representan la densidad de eventos o documentos mediante gradientes de color, útiles cuando el número de puntos es muy alto y su ubicación exacta es menos relevante que su concentración.

La Figura 4 muestra esquemáticamente estos tres tipos de mapa.

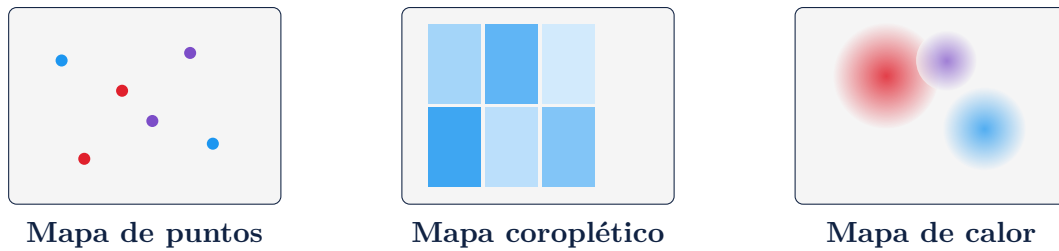


Figura 4: Tipos de representación cartográfica según la naturaleza de los datos geoespaciales: mapas de puntos, mapas coropléticos y mapas de calor.

B.4.2 Capas y zoom

Un mapa interactivo debe soportar múltiples niveles de detalle:

- **Capas:** organizar la información geoespacial en capas independientes que el usuario pueda activar o desactivar (por ejemplo, una capa por fenómeno, o una capa de fronteras administrativas sobre otra de eventos).
- **Niveles de zoom:** ajustar el nivel de agregación territorial visible según el nivel de acercamiento (por ejemplo, mostrar agregados por país al alejar el mapa y por municipio al acercarlo), similar a la jerarquía de zoom presente en los archivos PBF entregados en la Etapa 1.

B.5 Líneas de Tiempo

B.5.1 Líneas de tiempo basadas en fenómenos

Una línea de tiempo permite observar cómo evoluciona la cantidad o relevancia de los documentos asociados a cada fenómeno a lo largo del periodo cubierto por el corpus. Se recomienda construirla a partir de las categorías temporales obtenidas en la Sección B.1.2, diferenciando cada fenómeno mediante color u otra codificación visual consistente con el resto del tablero.

B.5.2 Evolución y reaparición de eventos

Además de la tendencia agregada por fenómeno, es útil identificar sobre la línea de tiempo momentos específicos en los que una misma entidad o un mismo evento reaparece en el corpus (por ejemplo, un hito legislativo, un incidente espacial o una crisis territorial mencionados en documentos de fechas distintas). Esto puede lograrse marcando sobre la línea de tiempo los `doc_id` en los que una entidad del grafo de conocimiento es mencionada, permitiendo distinguir entre un evento aislado y uno recurrente.

La Figura 5 ilustra ambos elementos: la codificación por color de cada fenómeno y el marcado de la reaparición de una misma entidad a lo largo del tiempo.

B.6 Dashboards Interactivos

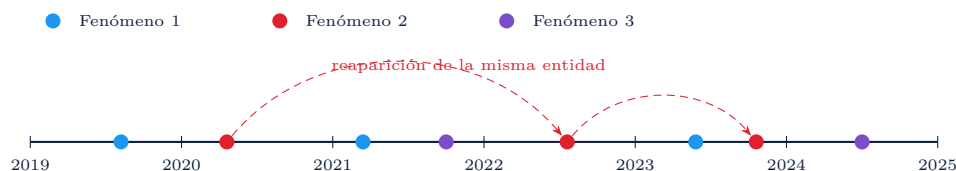


Figura 5: Línea de tiempo con eventos codificados por fenómeno y reaparición de una misma entidad a lo largo del corpus.

B.6.1 Tipos de layouts

Un tablero agrupa varias visualizaciones en una sola vista coherente. Entre las disposiciones más comunes se encuentran el layout de **cuadrícula** (paneles de tamaño fijo distribuidos en filas y columnas), el layout **maestro-detalle** (una vista principal acompañada de paneles secundarios que se actualizan según la selección en la vista principal) y el layout de **narrativa guiada** (una secuencia de vistas que conduce al usuario a través de un argumento analítico, útil para presentar hallazgos frente al equipo de expertos).

La Figura 6 esquematiza estas tres disposiciones.

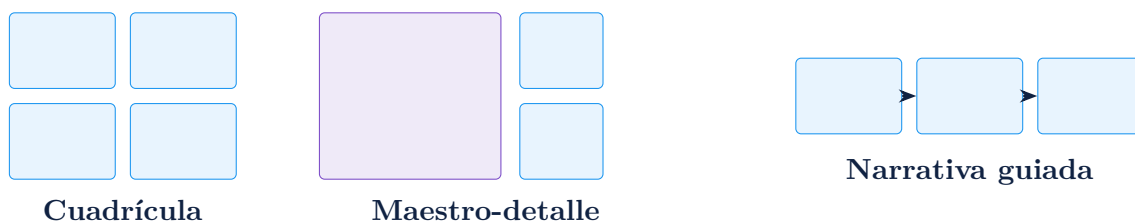


Figura 6: Disposiciones (*layouts*) habituales para organizar un tablero interactivo: cuadrícula, maestro-detalle y narrativa guiada.

B.6.2 Componentes y vistas

Un tablero para este reto debería integrar, como mínimo, una vista por cada una de las capacidades descritas en las secciones anteriores: comparación/distribución/-composición a nivel de metadata (Sección B.2), relaciones del grafo de conocimiento (Sección B.3), ubicación geoespacial (Sección B.4) y evolución temporal (Sección B.5), además de un mecanismo de acceso al texto original de los documentos y fragmentos que sustentan cada visualización.

B.6.3 Interactividad y filtros multi-vista

El valor de un tablero frente a un conjunto de gráficos aislados radica en su **coordinación**: una interacción del usuario en una vista debe poder afectar el estado de las demás. Se recomienda implementar filtros globales (por ejemplo, por fenómeno o por rango de fechas) que se propaguen a todas las vistas del tablero, así como interacciones de *brushing and linking*, en las que seleccionar un subconjunto de datos en una vista resalta automáticamente los elementos correspondientes en las demás vistas.

B.6.4 Arquitectura de una solución visual

B.6.4.1 Backend

El backend de la solución visual es responsable de exponer la metadata, el grafo de conocimiento y, de aplicar, el componente conversacional construidos en la Etapa 1 hacia el frontend de visualización, típicamente mediante una API que resuelva consultas de agregación, filtrado y búsqueda sin exponer directamente los archivos crudos del índice FAISS o del almacén de metadata.

B.6.4.2 Frontend

El frontend es responsable de renderizar las visualizaciones descritas en este documento y de gestionar la interacción del usuario (filtros, selección, exploración de vecinos, zoom), traduciendo dichas interacciones en solicitudes al backend.

B.6.4.3 Ciclo de vida del dato

De extremo a extremo, el dato recorre las siguientes etapas dentro de la solución visual: extracción desde la base vectorial y el grafo de conocimiento de la Etapa 1 (Sección B.1), enriquecimiento y normalización, exposición a través del backend (Sección B.6.4.1), y finalmente renderizado e interacción en el frontend (Sección B.6.4.2). Mantener esta trazabilidad de extremo a extremo es lo que permite que cualquier dato mostrado en el tablero pueda rastrearse de vuelta a su `doc_id` y `chunk_id` de origen. La Figura 7 resume este flujo de extremo a extremo.

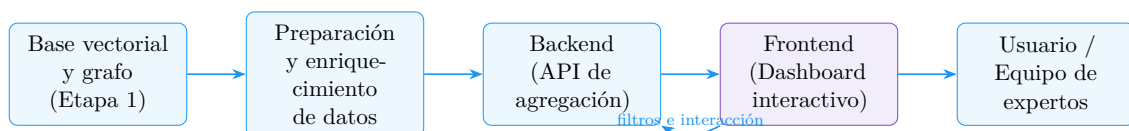


Figura 7: Arquitectura de extremo a extremo de la solución visual: del dato crudo de la Etapa 1 a la interacción del usuario con el tablero.